

CloudOps AI - Presentation Guide

A practical explanation of what it is, how to use it, its commands, safety features, supported providers, and the files that matter most.

1. What CloudOps AI is

CloudOps AI is not one AI model. It is a TypeScript command-line AI agent for Cloud and DevOps work. It can use several model providers, including OpenAI, Anthropic, Google Gemini, xAI Grok, Groq, Perplexity, and compatible custom APIs.

It accepts plain-English requests, builds a safe plan, calls approved cloud tools, checks results, and gives the user a concise response. The language model does not directly control the terminal: the application enforces permissions in code.

2. What it can do

Cloud inspection: inspect AWS accounts, S3 buckets, EC2 instances, Lambda functions, IAM summaries, CloudWatch alarms, VPCs, Cloudflare zones, Vercel projects, deployments, domains, GitHub repositories, pull requests, branches, and workflows.

Planning: create a step-by-step plan before taking action.

Safe execution: validate tools and commands before they run.

Verification: assess whether the requested task was completed.

Deployment: clone a GitHub, GitLab, or Bitbucket repository and deploy it to Vercel.

3. Install and start

Requirements: Node.js 20 or newer, npm, at least one AI provider API key, and the optional cloud CLIs you want to use (aws, gh, vercel, or wrangler).

Commands:

```
cd /home/ubuntu/cookie
npm install
cp .env.example .env
npm run build
npm start
```

Add an API key to .env before starting, for example: GROQ_API_KEY=your_key_here.
npm start opens interactive mode. Type a request, then type exit or quit to leave.

4. Everyday use

Use interactive mode:

```
npm start
```

```
> inspect my AWS infrastructure
```

Or send one request directly:

```
cloudops "inspect my AWS infrastructure"
```

```
npm run dev -- "list my Vercel projects"
```

Good requests are specific. Examples: "List EC2 instances and flag stopped ones", "Show failed GitHub Actions runs", and "Inspect recent Vercel deployments".

5. The agent workflow

User request -> intent and context -> plan -> select model -> request a tool call

-> validate inputs and permissions -> execute or simulate -> observe results

-> verify -> final response.

When --plan is used, CloudOps first generates a human-readable plan. It then runs an agent loop that may call tools until the task is complete or the iteration limit is reached. Finally, the verifier produces a completion assessment.

6. Main CLI command

Syntax: cloudops [options] [prompt]

cloudops "request" Send one natural-language request to the agent.

cloudops Start interactive chat mode.

--plan Generate and display a plan before executing.

--dry-run Simulate modifying operations; make no real changes.

--provider <id> Force an AI provider for this request.

--model <name> Force a particular model for this request.

-c, --config <path> Use a selected YAML or JSON configuration file.

-v, --verbose Print internal progress and debug information.

--version Print the application version.

--help Print command help.

7. Main CLI examples

```
cloudops --plan "diagnose my failing Vercel deployment"
```

```
cloudops --dry-run "deploy my application to production"
```

```
cloudops --verbose "show recent Vercel deployments"
```

```
cloudops --config ./production-config.yaml "inspect my infrastructure"
```

```
cloudops --provider anthropic --model claude-sonnet-4-20250514
```

```
"analyze my cloud architecture"
```

Use `--dry-run` for demonstrations. It lets you show the intended write action without changing cloud infrastructure.

8. Repository deployment command

Syntax: `cloudops deploy <repository> [options]`

`cloudops deploy https://github.com/owner/repository.git`

The command validates a supported Git URL, shallow-clones it to a temporary folder, runs `vercel deploy`, prints the public URL, and removes the temporary clone.

`--preview` Create a preview deployment instead of production.
`--branch <name>` Deploy a named Git branch.
`--project <name>` Target a Vercel project name.
`--team <slug>` Deploy under a Vercel team/scope.
`--token <token>` Provide a token; otherwise use `VERCEL_TOKEN`.
`--dry-run` Show the operation without cloning or deploying.

9. Deployment examples

`cloudops deploy https://github.com/owner/repository.git --preview`
`cloudops deploy https://github.com/owner/repository.git --branch staging`
`--project website --team acme`
`cloudops deploy https://github.com/owner/repository.git --dry-run`

10. Providers and model routing

Provider IDs: `openai`, `anthropic`, `gemini`, `xai`, `groq`, `perplexity`, and `custom`.

The configuration can choose a provider/model by task type: `default`, `coding`, `research`, `planning`, `verification`, and `fallback`. The example configuration uses Groq for default tasks, Anthropic for coding, Perplexity for research, OpenAI for planning and verification, and Gemini as a fallback.

The router considers task type, available API keys, command-line overrides, and fallback settings. Selecting `--provider` explicitly disables automatic fallback.

11. Configuration and API keys

Settings resolve in this order: command-line flags, environment variables, then `cloudops.config.yaml`. Important `.env` variables include:

`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY`, `XAI_API_KEY`, `GROQ_API_KEY`, `PERPLEXITY_API_KEY`, `CUSTOM_AI_BASE_URL`, `CUSTOM_AI_API_KEY`, `CUSTOM_AI_MODEL`.

Other useful settings: CLOUDOPS_DEFAULT_PROVIDER, CLOUDOPS_DEFAULT_MODEL, CLOUDOPS_FALLBACK_ENABLED, CLOUDOPS_SECURITY_MODE, CLOUDOPS_CONFIRM_DESTRUCTIVE, CLOUDOPS_CONFIRM_WRITE, and CLOUDOPS_LOG_LEVEL.

12. Tools exposed to the AI

AWS: sts_get_caller_identity, s3_list_buckets, ec2_describe_instances, lambda_list_functions, iam_get_account_summary, cloudwatch_describe_alarms, and ec2_describe_vpcs.

Vercel: list_projects, list_deployments, inspect_project, list_domains.

Cloudflare: account_info, list_zones.

GitHub: list_repos, view_repo, list_prs, list_workflows, view_run, search_code, and list_branches.

Shell: validated execution of allowlisted DevOps command-line programs.

13. Security model

The allowed command binaries are: aws, gh, git, vercel, wrangler, cloudflared, kubectl, terraform, npm, and node. Other binaries are rejected.

Read operations (for example aws s3 ls) are automatically allowed. Write operations (for example vercel deploy or git push) can require confirmation. Destructive operations (for example deleting a bucket, terminating instances, or force-pushing) require confirmation by default. --dry-run simulates non-read operations.

Secret redaction removes common API keys, AWS keys, GitHub tokens, JWTs, and key/value secrets before information goes to logs or AI context.

14. Security modes

strict: read operations allowed; write operations blocked; destructive work remains subject to confirmation policy.

standard: reads allowed; writes follow confirmation settings; destructive operations require confirmation by default.

permissive: reads and writes allowed; destructive confirmation can be disabled only through explicit configuration.

Presentation takeaway: safety is enforced outside the model using validation, permissions, confirmations, dry-run simulation, audit logging, and redaction.

15. Developer commands

`npm run build` Compile TypeScript into dist/.
`npm run dev` Run the CLI from TypeScript source with tsx.
`npm start` Run the compiled command-line interface.
`npm test` Run Vitest automated tests.
`npm run typecheck` Check TypeScript types without emitting files.
`npm run lint` Run ESLint checks.
`npm run lint:fix` Apply safe lint fixes where available.
`npm run format` Format source and tests with Prettier.
`npm run format:check` Check formatting without changing files.
`npm run check` Run type-checking, linting, and tests together.

16. Important files (read these last)

`README.md` High-level overview, setup, and examples.
`src/cli/index.ts` Every user-facing command and CLI option.
`cloudops.config.yaml` Routing, tools, security, and agent limits.
`.env.example` API keys and environment settings.
`src/agent/Agent.ts` Core plan -> tools -> verification loop.
`src/ai/ModelRouter.ts` Model selection and fallback behavior.
`src/tools/` Tool registration, permissions, confirmation.
`src/shell/CommandValidator.ts` Command allowlist and risk classification.
`src/deploy/VercelDeployment.ts` Repository-to-Vercel deployment workflow.
`docs/architecture.md` Architecture diagram and extension approach.
`docs/security.md` Threat model and safeguards.